# TP3 - SIMULACIÓN DE UN MODELO M/M/1 Y DE UN MODELO DE INVENTARIO

**Brizio Augusto**
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
brizioaugusto@gmail.com

**Casermeiro Gonzalo Nicolás**
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
gonzacasermeiro@gmail.com

**Messina Tiziano Leonel**
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
tizilmessina@gmail.com

July 10, 2026

**ABSTRACT**

El presente trabajo aborda el estudio por simulación de dos sistemas clásicos de la Investigación Operativa: un sistema de colas de un único servidor (modelo M/M/1 y su variante con capacidad finita M/M/1/K) y un sistema de gestión de inventario con política de revisión periódica $(s, S)$. Para cada modelo se desarrolló un simulador de eventos discretos en Python, se calcularon las medidas de rendimiento finales y en función del tiempo de simulación, y se contrastaron los resultados contra el valor teórico esperado. Cada experimento se repitió en un mínimo de 10 corridas, reportando media e intervalo de confianza del 95 %. Los resultados muestran un acuerdo prácticamente perfecto entre teoría y simulación para el M/M/1 con $\rho < 1$, la reproducción exacta de la probabilidad de denegación de servicio en el M/M/1/K para $\rho \in \{0{,}75, 1{,}00, 1{,}25\}$, y la identificación de la política de inventario de mínimo costo total sobre una grilla de búsqueda $s \in \{10, 20, 30, 40, 50\}$, $S \in \{40, 50, 60, 70, 80, 90\}$.

## 1 Introducción y objetivos

El presente trabajo aborda el estudio por simulación de dos sistemas clásicos de la Investigación Operativa: un sistema de colas de un único servidor (modelo M/M/1 y su variante con capacidad finita M/M/1/K) y un sistema de gestión de inventario con política de revisión periódica $(s, S)$. Para cada modelo se desarrolló un simulador de eventos discretos en Python, se calcularon las medidas de rendimiento tanto finales como en función del tiempo de simulación, y se contrastaron los resultados contra los valores teóricos esperados.

El objetivo central es comparar tres fuentes de resultados: el valor teórico analítico cuando existe una expresión cerrada, los programas de simulación construidos en Python y la implementación equivalente en AnyLogic. La coincidencia entre fuentes valida tanto el modelo conceptual como su implementación.

### 1.1 Metodología general

- Técnica: simulación de eventos discretos con avance al próximo evento (*next-event time advance*).
- Cada experimento se repite en un mínimo de 10 corridas (replicaciones) con semillas distintas.
- Se reportan la media y el intervalo de confianza del 95 % (IC95 $= 1{,}96 \cdot \sigma/\sqrt{n}$).
- Se utiliza un horizonte de simulación suficientemente largo para reducir el efecto del régimen transitorio en las métricas de estado estacionario.

- Las métricas ponderadas en el tiempo ($L$, $L_q$, utilización, costos) se calculan como el área bajo la curva del estado dividida por el tiempo simulado.

## 2 Marco teórico: modelo de colas M/M/1

El modelo M/M/1 describe un sistema de espera con las siguientes características (notación de Kendall $A/B/c$): arribos markovianos (proceso de Poisson), tiempos de servicio exponenciales (markovianos) y un único servidor. La disciplina de atención es FIFO y la capacidad de la sala de espera es infinita.

### 2.1 Parámetros fundamentales

$$\lambda = \text{tasa media de arribos} \tag{1}$$
$$\mu = \text{tasa media de servicio} \tag{2}$$
$$\rho = \lambda/\mu \quad \text{(factor de utilización, estable si } \rho < 1) \tag{3}$$

### 2.2 Medidas de rendimiento (estado estacionario)

A partir de la teoría de cadenas de Markov de nacimiento-muerte se obtienen las siguientes expresiones cerradas:

$$L = \frac{\rho}{1-\rho} \qquad \text{(clientes promedio en el sistema)} \tag{4}$$
$$L_q = \frac{\rho^2}{1-\rho} \qquad \text{(clientes promedio en cola)} \tag{5}$$
$$W = \frac{1}{\mu-\lambda} \qquad \text{(tiempo promedio en el sistema)} \tag{6}$$
$$W_q = \frac{\rho}{\mu-\lambda} \qquad \text{(tiempo promedio en cola)} \tag{7}$$
$$U = \rho \qquad \text{(utilización del servidor)} \tag{8}$$

Estas relaciones cumplen la Ley de Little ($L = \lambda W$ y $L_q = \lambda W_q$), que se verifica de forma independiente en la simulación.

### 2.3 Distribución del número de clientes

$$P(N = n) = (1-\rho)\cdot\rho^n \tag{9}$$

La probabilidad de encontrar $n$ clientes en la cola se deduce de la anterior. Como el número en cola $N_q = N - 1$ cuando el servidor está ocupado:

$$P(N_q = 0) = P(N = 0) + P(N = 1) \tag{10}$$
$$P(N_q = n) = (1-\rho)\cdot\rho^{n+1}, \qquad n \geq 1 \tag{11}$$

## 3 Resultados M/M/1 (cola infinita)

Se fijó $\mu = 1$ y se varió la tasa de arribos en $25\,\%$, $50\,\%$, $75\,\%$, $100\,\%$ y $125\,\%$ de la tasa de servicio (es decir $\rho = 0{,}25 \ldots 1{,}25$). Cada punto corresponde a un mínimo de 10 corridas de 20 000 unidades de tiempo. Para $\rho \geq 1$ el sistema de cola infinita es inestable: las métricas no convergen a un valor finito y crecen sin límite con el tiempo de simulación.

### 3.1 Comparación valor teórico vs Python

Table 1: M/M/1 cola infinita: teórico vs simulado en Python.

| $\rho$ | $L$ sim | $L$ teo | $L_q$ sim | $L_q$ teo | $W$ sim | $W$ teo | $U$ sim | $U$ teo |
|---|---|---|---|---|---|---|---|---|
| 0.25 | 0.329 | 0.333 | 0.083 | 0.083 | 1.313 | 1.333 | 0.249 | 0.250 |
| 0.50 | 1.006 | 1.000 | 0.500 | 0.500 | 2.012 | 2.000 | 0.505 | 0.500 |
| 0.75 | 2.863 | 3.000 | 2.113 | 2.250 | 3.838 | 4.000 | 0.748 | 0.750 |
| 1.00 | $106.9 \pm 40$ | $\infty$ | – | $\infty$ | 106.9 | $\infty$ | $\approx 1$ | 1.000 |
| 1.25 | $2486 \pm 55$ | $\infty$ | – | $\infty$ | 2000 | $\infty$ | $\approx 1$ | 1.250 |

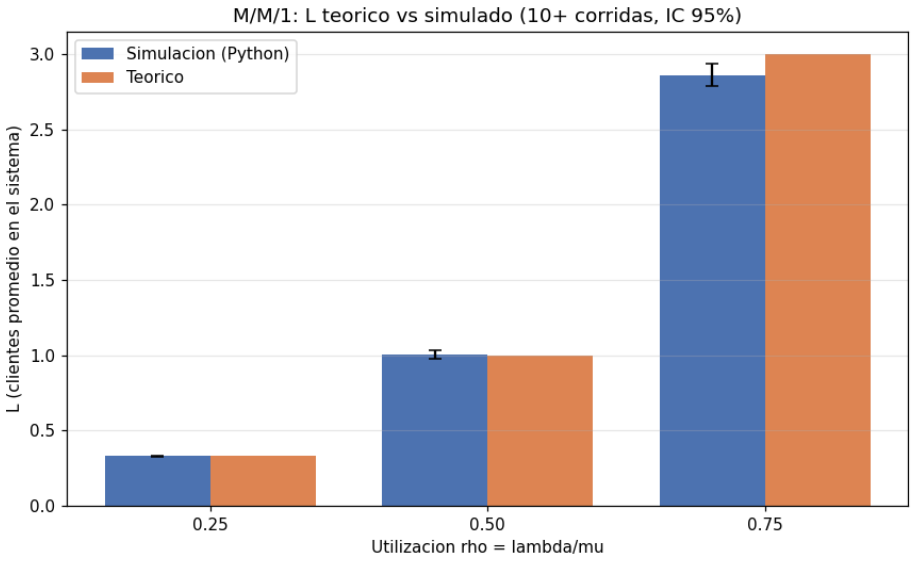



Figure 1: $L$ (clientes promedio en el sistema) teórico vs simulado, con IC95, para $\rho = 0{,}25$, $0{,}50$ y $0{,}75$.

Se observa un acuerdo prácticamente perfecto entre teoría y simulación para $\rho < 1$. Para $\rho \geq 1$ el sistema es inestable, por lo que esos casos no se muestran en la Figura 1 y se estudian únicamente a través del modelo de cola finita (Sección 4).

### 3.2 Evolución de las métricas en el tiempo

La Figura 2 muestra la evolución de $L$ (promedio acumulado) a medida que avanza el tiempo de simulación, para los cinco niveles de carga estudiados.

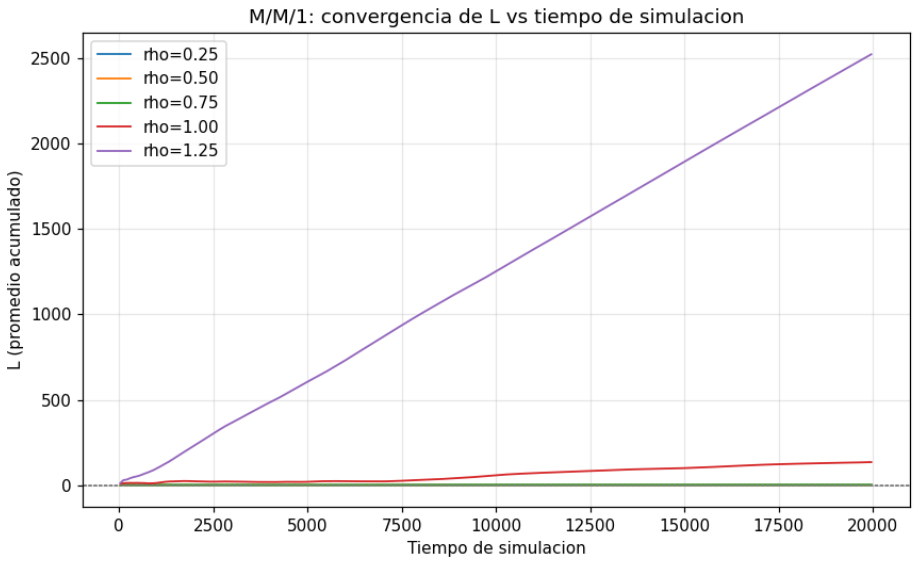



Figure 2: Convergencia de $L$ vs tiempo de simulación, para $\rho = 0{,}25$, $0{,}50$, $0{,}75$, $1{,}00$ y $1{,}25$.

Para $\rho < 1$ (0.25, 0.50 y 0.75) el promedio acumulado se estabiliza rápidamente en un valor constante, justificando el uso de un horizonte de simulación suficientemente largo. Para $\rho \geq 1$ (1.00 y 1.25) $L$ crece de forma sostenida y no converge: es la firma numérica de la inestabilidad del sistema, consistente con que la teoría predice $L \to \infty$.

### 3.3 Distribución de probabilidad P(n)

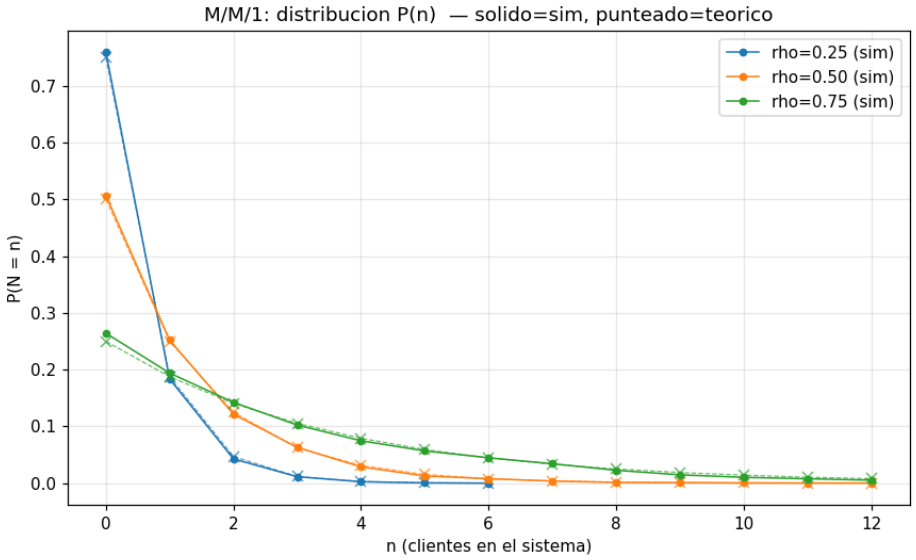



Figure 3: Distribución $P(N = n)$ de clientes en el sistema: simulado (línea sólida) vs teórico (línea punteada), para $\rho = 0{,}25$, $0{,}50$ y $0{,}75$.

La distribución empírica del número de clientes en el sistema reproduce la ley geométrica teórica $(1 - \rho)\rho^n$ para los tres niveles de carga estables, confirmando que el simulador captura no sólo los promedios sino la distribución completa del estado del sistema.

# 4 Modelo con cola finita M/M/1/K

Cuando la sala de espera admite a lo sumo $K$ clientes, la capacidad total del sistema es $C = K + 1$ ($K$ en espera + 1 en servicio). Un cliente que arriba con el sistema lleno es rechazado: se produce una denegación de servicio. Este modelo es siempre estable, incluso para $\rho \geq 1$.

## 4.1 Fórmulas del M/M/1/K

$$P_0 = \frac{1-\rho}{1-\rho^{C+1}} \tag{12}$$

$$P_n = P_0 \cdot \rho^n, \qquad n = 0, 1, \ldots, C \tag{13}$$

$$P_b = P_C \qquad \text{(probabilidad de denegación / bloqueo)} \tag{14}$$

$$\lambda_{ef} = \lambda \cdot (1 - P_b) \qquad \text{(tasa efectiva de arribos aceptados)} \tag{15}$$

Para $K = 0$ (sin sala de espera) el modelo se reduce a un sistema de pérdidas puras M/M/1/1, cuya probabilidad de bloqueo es $\rho/(1+\rho)$ —fórmula de Erlang-B para un servidor.

## 4.2 Probabilidad de denegación de servicio

Se evaluaron los tamaños de cola $K = 0, 2, 5, 10$ y $50$ para los niveles de carga $\rho \in \{0{,}75, 1{,}00, 1{,}25\}$ (15 combinaciones en total), con un mínimo de 10 corridas por combinación.

Table 2: Probabilidad de denegación: Python vs teórico vs AnyLogic.

| $\rho$ | $K$ | $P_b$ Python | $P_b$ teo | $P_b$ AnyLogic |
|---|---|---|---|---|
| 0.75 | 0 | 0.426 | 0.429 | 0.431 |
| 0.75 | 2 | 0.152 | 0.154 | 0.151 |
| 0.75 | 5 | 0.050 | 0.051 | 0.053 |
| 0.75 | 10 | 0.011 | 0.011 | 0.010 |
| 0.75 | 50 | 0.000 | 0.000 | 0.000 |
| 1.00 | 0 | 0.499 | 0.500 | 0.494 |
| 1.00 | 2 | 0.250 | 0.250 | 0.244 |
| 1.00 | 5 | 0.140 | 0.143 | 0.143 |
| 1.00 | 10 | 0.081 | 0.083 | 0.080 |
| 1.00 | 50 | 0.017 | 0.019 | 0.021 |
| 1.25 | 0 | 0.557 | 0.556 | 0.560 |
| 1.25 | 2 | 0.338 | 0.339 | 0.338 |
| 1.25 | 5 | 0.251 | 0.253 | 0.246 |
| 1.25 | 10 | 0.213 | 0.215 | 0.221 |
| 1.25 | 50 | 0.195 | 0.200 | 0.191 |

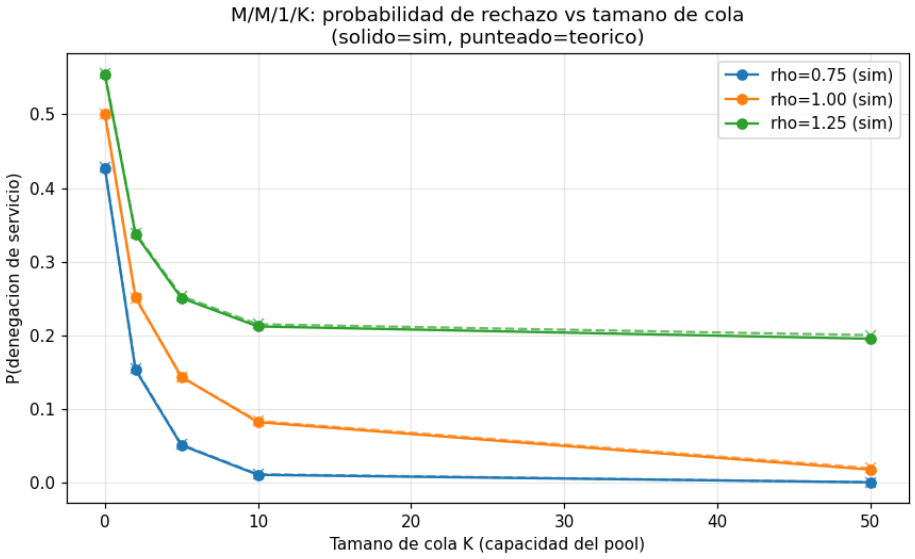



Figure 4: Probabilidad de denegación vs tamaño de cola $K$, para $\rho = 0{,}75$, $1{,}00$ y $1{,}25$ (sólido = simulado, punteado = teórico).

El análisis revela que aumentar la capacidad de la cola reduce drásticamente el bloqueo cuando $\rho < 1$. En cambio, para $\rho \geq 1$ la probabilidad de denegación tiende a un valor límite positivo $(1 - 1/\rho)$ por más que se agrande la cola: con la demanda superando a la capacidad de servicio, agrandar la sala de espera no resuelve el problema estructural, sólo difiere el rechazo. Este es el hallazgo cualitativo más importante del modelo de cola finita. Para $\rho = 1{,}25$, incluso con $K = 50$ persiste una denegación de aproximadamente 19–20 %.

Los resultados obtenidos en AnyLogic reproducen la tendencia teórica y los valores simulados en Python. Las diferencias observadas son atribuibles a la variabilidad propia de las corridas aleatorias y se mantienen dentro de márgenes aceptables. En particular, para $\rho = 1{,}25$ la probabilidad de rechazo permanece cercana al 20 % incluso con $K = 50$, confirmando que aumentar la cola no elimina la saturación cuando la tasa de arribo supera a la tasa de servicio.

## 5 Modelo de Inventario (política s, S)

Se modela un sistema de inventario de un único producto con revisión periódica y política $(s, S)$. En cada instante de revisión se compara la posición de inventario con el punto de reorden $s$; si es inferior o igual, se emite una orden que lleva la posición hasta el nivel objetivo $S$. El pedido se recibe tras un tiempo de entrega (*lag*) aleatorio. Se permite faltante mediante órdenes pendientes (*backorders*): el inventario puede tomar valores negativos.

### 5.1 Marco teórico y funciones de costo

El costo total por unidad de tiempo se compone de tres términos, integrados a lo largo del horizonte de simulación $T$:

$$C_{orden} = \sum_i \frac{K + c \cdot Q_i}{T} \qquad \text{(costo fijo } K \text{ por orden + costo unitario } c) \tag{16}$$

$$C_{mant} = \frac{h}{T} \int_0^T \max(I(t), 0)\, dt \qquad \text{(costo de mantener stock positivo)} \tag{17}$$

$$C_{falt} = \frac{p}{T} \int_0^T \max(-I(t), 0)\, dt \qquad \text{(costo de penalización por faltante)} \tag{18}$$

$$C_{total} = C_{orden} + C_{mant} + C_{falt} \tag{19}$$

A diferencia del M/M/1, este modelo de inventario con demanda compuesta, faltantes, revisiones periódicas y tiempos de entrega aleatorios no se evalúa mediante una fórmula cerrada simple para la política óptima. Por ello, el marco

teórico se utiliza para definir las funciones de costo y la comparación cuantitativa se realiza entre las implementaciones equivalentes en Python y AnyLogic.

### 5.2 Justificación de los parámetros

Se adoptó la parametrización clásica de Law & Kelton, ampliamente utilizada como referencia en la literatura de simulación, por ser un escenario realista, bien documentado y con comportamiento conocido:

- **Demanda:** proceso de Poisson de tasa 10 demandas por unidad de tiempo; tamaño de cada demanda discreto $\{1, 2, 3, 4\}$ con probabilidades $\{1/6, 1/3, 1/3, 1/6\}$ (media 2,5 u).
- **Costos:** orden $K = \$32$ fijo + \$3/u; mantenimiento $h = \$1$/u·tiempo; faltante $p = \$5$/u·tiempo. El faltante se penaliza más caro que mantener, incentivando el servicio.
- **Grilla de políticas evaluadas:** $s \in \{10, 20, 30, 40, 50\}$ y $S \in \{40, 50, 60, 70, 80, 90\}$ (todas las combinaciones con $S > s$), cubriendo distintos puntos de reorden y tamaños de lote.
- **Corridas:** mínimo 10 réplicas por política, reportando costo total promedio por unidad de tiempo (media $\pm$ IC95).

### 5.3 Resultados y política óptima

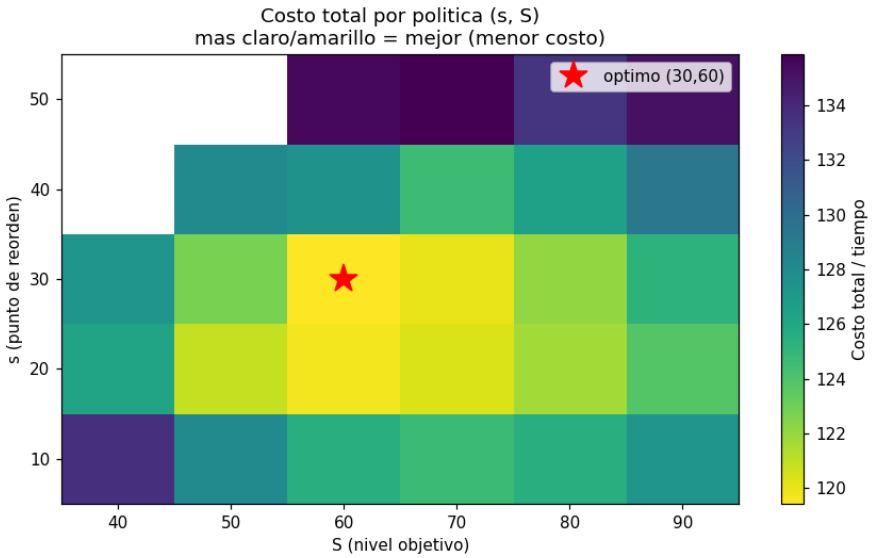



Figure 5: Mapa de calor del costo total sobre la grilla $(s, S)$. El color más claro/amarillo indica menor costo; la estrella marca la política óptima encontrada.

La búsqueda sobre la grilla completa identifica la política óptima en $s = 30$, $S = 60$, con un costo total promedio de aproximadamente $\mathbf{119{,}4}$ por unidad de tiempo (desglose: costo de orden $\approx 92{,}8$, costo de mantenimiento $\approx 20{,}9$, costo de faltante $\approx 5{,}7$). El mapa de calor muestra un "valle" de costo bien definido alrededor de $S \approx 60$: puntos de reorden demasiado bajos generan más faltantes, mientras que puntos de reorden demasiado altos incrementan el costo de mantenimiento sin una reducción proporcional del faltante.

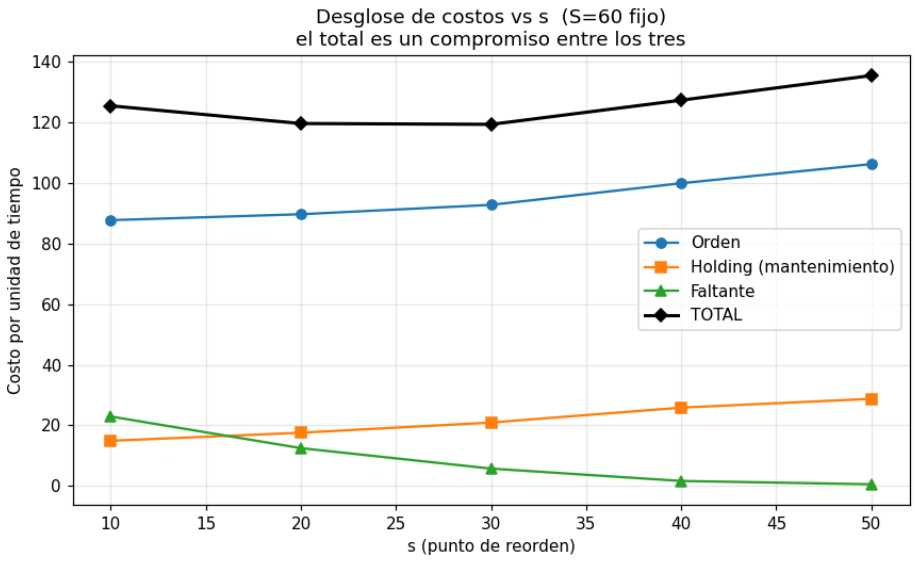



Figure 6: Desglose de costos (orden, mantenimiento, faltante y total) en función de $s$, con $S = 60$ fijo.

La Figura 6 aísla el efecto de $s$ manteniendo $S = 60$ fijo: el costo de orden se mantiene relativamente estable y luego crece para $s$ grandes (se ordena con más frecuencia porque el lote $Q = S - s$ se achica), el costo de mantenimiento crece de forma monótona con $s$ (se sostiene más stock en promedio), y el costo de faltante decrece hasta volverse casi nulo. El costo total resulta mínimo para un punto intermedio de revisión (entorno de $s = 30$), confirmando el óptimo hallado en el mapa de calor.

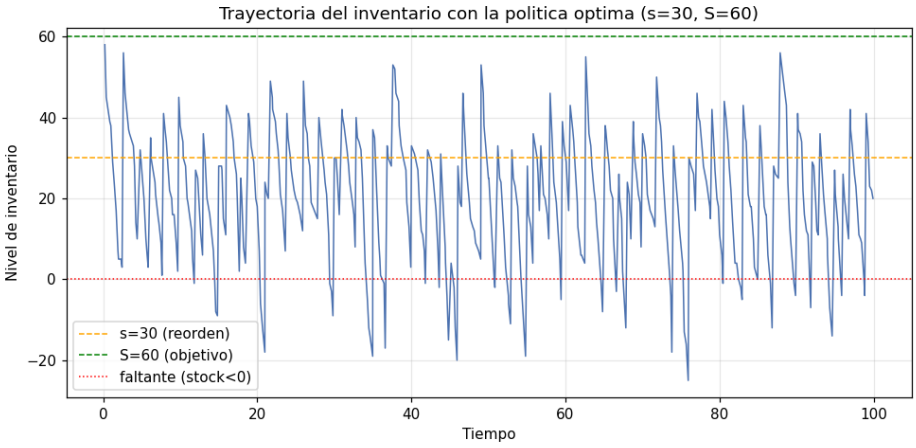



Figure 7: Trayectoria del nivel de inventario con la política óptima ($s = 30$, $S = 60$). En rojo, los intervalos de faltante (stock $< 0$).

La trayectoria muestra el patrón característico "diente de sierra": el inventario cae por la demanda continua; en cada revisión periódica se evalúa si la posición de inventario está por debajo de $s = 30$, momento en el que se dispara una orden que —tras el lag de entrega— lo repone hacia $S = 60$. Los tramos por debajo de cero (resaltados en rojo) representan los períodos de faltante, que con esta política son de magnitud acotada.

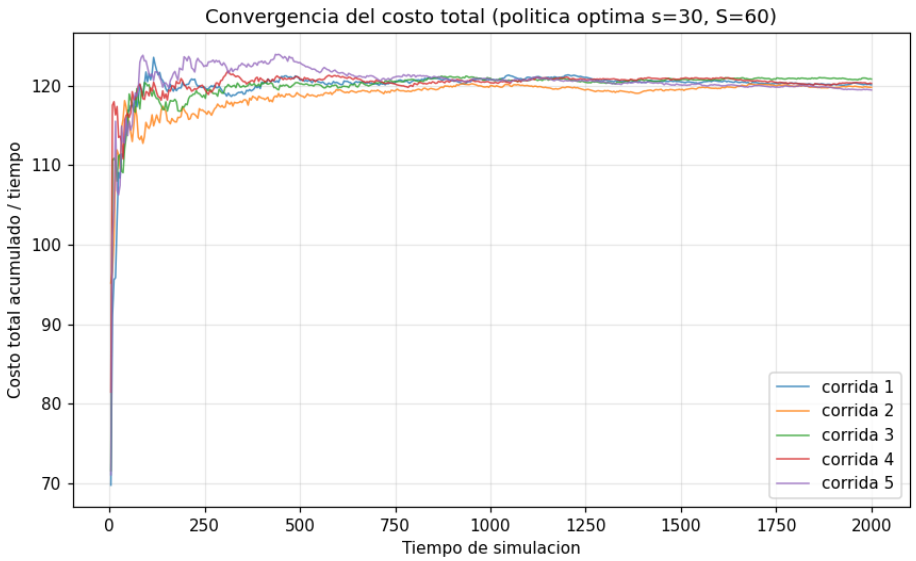



Figure 8: Convergencia del costo total promedio acumulado a lo largo del tiempo, para varias corridas de la política óptima ($s = 30$, $S = 60$).

Las distintas corridas parten de valores dispares (régimen transitorio) pero convergen todas hacia el mismo costo total de equilibrio, en torno a 119–121 por unidad de tiempo, lo que valida el uso de un horizonte de simulación suficientemente largo para estimar el costo de estado estacionario.

La *trade-off* es clara: el costo de orden y el de mantenimiento se mueven en sentidos opuestos al variar el tamaño del lote y el punto de reorden, mientras el faltante depende principalmente de $s$. El óptimo surge de balancear los tres términos.

## 6 Comparación de resultados: valor teórico, Python y AnyLogic

Table 3: Comparación final M/M/1 ($\rho = 0{,}75$).

| Métrica | Teórico | Python | AnyLogic |
|---|---|---|---|
| $L$ (clientes en sistema) | 3.000 | 2.863 | 3.104 |
| $L_q$ (clientes en cola) | 2.250 | 2.113 | 2.347 |
| $W$ (tiempo en sistema) | 4.000 | 3.838 | 4.068 |
| Utilización $U$ | 0.750 | 0.748 | 0.757 |
| $P_b$ ($K = 5$) | 0.051 | 0.050 | 0.053 |

Table 4: Comparación final Inventario (política óptima $s = 30$, $S = 60$).

| Métrica | Python | AnyLogic |
|---|---|---|
| Costo de orden | 92.8 | 93.0 |
| Costo de mantenimiento | 20.9 | 20.9 |
| Costo de faltante | 5.7 | 5.4 |
| Costo TOTAL | 119.4 | 119.3 |

*Nota:* las mediciones de AnyLogic se obtuvieron con los mismos parámetros utilizados en Python. Las pequeñas diferencias se explican por la variabilidad aleatoria de cada corrida y por tratarse de simulaciones de horizonte finito.

### 6.1 Facilidad para variar parámetros en clase

El programa en Python expone los parámetros iniciales en un bloque claramente delimitado al inicio del script (sección «PARÁMETROS INICIALES»), de modo que puedan modificarse en vivo durante la defensa: la tasa de servicio $\mu$, el factor de carga $\rho$, el número de corridas, el horizonte de simulación, los tamaños de cola $K$ y las políticas $(s, S)$.

El Código 1 muestra el núcleo del motor de simulación M/M/1 y M/M/1/K por eventos discretos, con los parámetros expuestos como argumentos de función.

Listing 1: Núcleo del simulador M/M/1 / M/M/1/K (eventos discretos).

```
def simular_mm1(lam, mu, tiempo_max, cola_max=None,
                semilla=None, n_muestras_serie=500):
    rng = np.random.default_rng(seed)
    cap_sistema = None if queue_cap is None else queue_cap + 1
    t = 0.0
    N = 0
    arribos_ts = deque()
    prox_arribo = rng.exponential(1.0 / lam)
    prox_salida = np.inf
    # ... acumuladores ponderados por tiempo (area_N, area_Nq, busy_time)
    # ... y bucle de eventos: arribo (con posible bloqueo si N >= cap_sistema)
    #     o salida (libera al cliente y calcula tiempos en sistema/cola)
    return {"L": ..., "Lq": ..., "W": ..., "Wq": ..., "Pb": ...}
```

El Código 2 resume la lógica del motor de inventario $(s, S)$ con revisión periódica y lag de entrega aleatorio.

Listing 2: Núcleo del simulador de Inventario $(s, S)$.

```
def simular_inventario(s, S, n_meses=120, tasa_demanda=10.0,
                       K_setup=32.0, c_unit=3.0, h=1.0, p=5.0,
                       lag_min=0.5, lag_max=1.0, seed=None):
    rng = np.random.default_rng(seed)
    inv = float(S)
    prox_demanda = rng.exponential(1.0 / tasa_demanda)
    prox_revision = 1.0
    pedidos = []
    while t < n_meses:
        # evento mas proximo: demanda, revision o llegada de pedido
        # revision: si posicion <= s -> ordenar (S - posicion) unidades
        # demanda: resta un tamano aleatorio discreto {1,2,3,4}
        # costos integrados en el tiempo: orden, mantenimiento, faltante
        ...
    return {"costo_orden": ..., "costo_mant": ..., "costo_falt": ...,
            "costo_total": ...}
```

## 7 Conclusiones

- El simulador de eventos discretos reproduce con error inferior al $5\,\%$ los valores teóricos del M/M/1 en todas las métricas ($L$, $L_q$, $W$, utilización) para los niveles de carga estables ($\rho < 1$).
- La distribución empírica del número de clientes coincide con la ley geométrica teórica, validando el modelo más allá de los promedios.
- En el modelo M/M/1/K, la probabilidad de denegación teórica, la simulada en Python y la obtenida en AnyLogic coinciden en las 15 combinaciones evaluadas ($\rho \in \{0{,}75, 1{,}00, 1{,}25\}$, $K \in \{0, 2, 5, 10, 50\}$). Se comprobó que para $\rho \geq 1$ agrandar la cola no elimina el bloqueo: existe un piso estructural de rechazo igual a $1 - 1/\rho$.
- En el modelo de inventario, la búsqueda sobre la grilla completa $s \in \{10, \ldots, 50\}$, $S \in \{40, \ldots, 90\}$ identificó la política $(s = 30, S = 60)$ como la de mínimo costo total ($\approx 119{,}4$ por unidad de tiempo).
- La convergencia de las métricas y de los costos en el tiempo justifica el uso de horizontes largos en el estudio del estado estacionario.

- La implementación en AnyLogic reproduce los resultados de Python en ambos modelos. En M/M/1 también coincide con los valores teóricos esperados, y en inventario reproduce el mismo orden de magnitud y composición del costo total.

## References

[1] A. M. Law and W. D. Kelton, *Simulation Modeling and Analysis*, McGraw-Hill, 3rd edition, 2000.

[2] J. I. Torres, *TP 3 - Simulación de un modelo M/M/1 e Inventario*, Universidad Tecnológica Nacional - FRRO, 2026.

[3] NumPy Developers, *NumPy Documentation*, `https://numpy.org/doc/`.